

Interview Prep: Axiom MVP

Scenario-based answers for life sciences, cloud, and AI conversations. Say these in your own words - you do not need to sound like an expert.

Quick Elevator Pitch (30 seconds)

"I built Axiom, an MVP for competitive pharmaceutical intelligence. A user types a question like 'What is the Alzheimer trial landscape?' and the app pulls live data from ClinicalTrials.gov, FDA, and PubMed, shows charts and rankings, and uses AI to explain what it found. I vibe-coded it as a fast proof-of-concept on Docker and a GCP VM, with guardrails so the AI explains real numbers instead of making them up."

The One Idea to Remember

- Problem: Teams waste time jumping between websites to understand a therapy area.
- Solution: One workspace that fetches live public data, visualizes it, and uses AI as an assistant.
- Why MVP: Prove the workflow works before investing in enterprise features.

Part 1: Core Questions and Answers

Scenario A: "Tell me about your project."

What to say:

Axiom is an AI-powered research workspace for life sciences competitive intelligence. Instead of opening five different websites, a user asks one question in plain English. The system searches live trial data, FDA safety info, and PubMed papers, then builds a dashboard - charts, trial tables, momentum rankings - and lets the user chat with an AI agent about what it found. They can also generate an executive briefing or run a bull-vs-bear investment debate on the data.

It is an MVP I built to show how cloud + AI can speed up early-stage research workflows. It is deployed on a Google Cloud VM with Docker and nginx.

If they want more detail, add:

- Frontend: Next.js (what users see)
- Backend: Python FastAPI (the brain)
- Database: PostgreSQL (saves investigations and user sessions)
- AI: OpenRouter with DeepSeek V4 Flash (open-source), using tool calling so the model fetches data instead of guessing

Scenario B: "Why did you choose this project?"

What to say - three reasons:

- It matches the industry. Life sciences teams need competitive landscape views - real work, not a toy demo.
- It shows cloud + AI together. The AI orchestrates live data fetches and explains results.
- Public data makes an MVP possible. ClinicalTrials.gov, openFDA, and PubMed are free official APIs.

Simple analogy:

I did not build a fake demo with hardcoded charts. I built a working assistant that goes to the real library every time.

Scenario C: "What problem does it solve?"

Today, competitive intelligence in pharma often looks like this: open ClinicalTrials.gov, export spreadsheets, search PubMed separately, check FDA adverse events elsewhere, then write a slide deck by hand. That takes hours and the data is stale the next day.

Axiom reduces that to one flow: ask a question, get a live dashboard, ask follow-ups, export a briefing. The AI handles the tedious gathering and first-pass synthesis. The human still makes decisions - but they start with organized, cited, live data.

Who benefits:

- BD / strategy teams scouting therapy areas
- Researchers comparing mechanisms
- Anyone doing early competitive landscaping before a deeper analyst dive

Scenario D: "You said you vibe-coded this. What does that mean?"

Vibe coding means I used AI coding tools - mainly Cursor and Antigravity - to build the MVP quickly. I described what I wanted, reviewed the code the AI suggested, tested it, and iterated. I did not hand-write every line from scratch.

That let me focus on product flow and architecture instead of boilerplate. For an MVP, speed matters. I own the design decisions - data sources, agent tools, dashboard layout - even if AI helped write the implementation.

If they push on quality:

I still validate behavior: does the agent cite real NCT IDs? Do charts match API data? Does deployment work on GCP? Vibe coding is an accelerator, not a substitute for thinking.

Tools you used:

- Cursor/Antigravity - main IDE; AI pair programmer for frontend, backend, Docker, fixes
- OpenRouter - API gateway to access LLMs with one integration
- DeepSeek V4 Flash - open-source model for agent reasoning, tool selection, explanations, briefings, debate
- Docker Compose - run database + API + web together
- GCP VM - host the app in the cloud
- nginx - front door, routes visitors to the right service
- dpdns - free domain pointing to your VM

Scenario E: "How does the AI actually work?"

What to say - step by step:

- User types: "Show me the diabetes drug trial landscape."
- Frontend sends that to the FastAPI backend.
- The orchestrator (agent loop) sends the question to the LLM with a list of tools it can call.
- The LLM decides which tools to call (e.g. `get_therapy_landscape`, `rank_therapies_by_momentum`).
- Backend runs those tools - they hit real APIs (ClinicalTrials.gov, PubMed, openFDA).
- Results go back to the LLM. It writes a summary using only that data.
- Backend saves the investigation, builds dashboard config, streams text to the UI.
- User can ask follow-ups or generate an executive briefing.

Key phrase:

The LLM is the coordinator. The numbers come from code and public APIs, not from the model's memory.

Scenario F: "How do you stop the AI from making up trial data?"

What to say - four guardrails:

- Tool-first architecture - agent must call tools to get trial counts, rankings, and safety data.
- Server-computed scores - momentum rankings are calculated in Python from live data.
- Strict prompts - never invent trial counts, drug names, or statistics; cite NCT IDs.

- Validation - code checks if synthesis contradicts empty data.

For production I would add: source citations in UI, audit logs, human review for high-stakes outputs.

Scenario G: "Why DeepSeek V4 Flash and OpenRouter?"

I use DeepSeek V4 Flash - an open-source model that is fast, cost-effective, and strong at coding, reasoning, and agent workflows. I access it through OpenRouter, which gives me one API integration, easy model swapping, and provider failover without rewriting the app. Under the hood it uses the same OpenAI-compatible API format with chat completions and function calling.

Part 2: Features - What Exists vs What You Could Add

What the MVP already has (name 2-3 in an interview)

- Live investigation - Fetches fresh trial/FDA/PubMed data per query
- AI agent with tools - LLM calls 8+ tools - search trials, landscape, rankings, etc.
- Dynamic dashboard - Charts and sections adapt to question intent
- Momentum rankings - Server-side scoring from live trials
- White-space opportunities - Finds undercrowded mechanism areas
- Competitive matrix - Bubble chart positioning therapies
- Explain these signals - AI narrates the dashboard data
- Follow-up chat - Ask more questions on same investigation
- Executive briefing - Markdown report from live investigation
- Bull vs Bear debate - Two AI analysts argue opposite sides, then synthesize
- Session library - Save and revisit past investigations
- Auth + admin - Login, Google OAuth option, token limits

Future features (answer "where would you take this?")

- 1. Proprietary data connectors - customer internal pipelines, CRM, lab data
- 2. Alerting - notify when a competitor starts a Phase 3 trial
- 3. Regulatory workflow - audit-ready reports with full source trail
- 4. Multi-tenant workspaces - teams, roles, shared investigations
- 5. RAG on internal documents - PDFs, SEC filings, conference abstracts
- 6. Forecasting module - approval timeline estimates from historical data
- 7. Comparator trials - side-by-side protocol comparison across sponsors
- 8. Safety signal monitoring - continuous openFDA watch with trend detection
- 9. Fine-tuned domain model - smaller model trained on pharma language
- 10. GCP-native scaling - Cloud Run, GKE, Cloud SQL, Secret Manager, IAM

How to say it simply:

The MVP proves the user journey. Production would add customer data, compliance, alerts, and enterprise cloud hardening - that is where a life sciences cloud company adds real value.

Part 3: "Where Are You Going Forward?"

Scenario H: "What is your roadmap?"

Phase 1 is done - working MVP with live public data and AI agent. Phase 2 would harden for real users: better auth, monitoring, faster caching, more data sources. Phase 3 would be enterprise: private data integration, team features, compliance, and deployment on proper GCP services instead of a single VM.

Phased breakdown:

- Now (MVP): Live APIs, agent, dashboard, briefing, deploy on GCP VM
- Next: Caching, error handling, mobile polish, more charts
- Later: SSO, audit logs, private data, alerts, multi-tenant, GxP considerations

Scenario I: "How is this relevant to our company?"

If they are life sciences focused:

It automates a workflow your customers already do manually - competitive landscaping. The MVP shows AI can sit on top of trusted public data sources and produce analyst-ready outputs.

If they are cloud focused:

It is a containerized app designed for cloud deploy. The next step is migrating from a single VM to managed GCP services - Cloud Run, Cloud SQL, load balancing.

If they are AI focused:

It is not a chatbot wrapper. It is an agentic system with tool calling, streaming, guardrails against hallucination, and structured outputs (dashboards, briefings, debate). That is a reusable pattern for other domain agents.

Part 4: Technical Scenarios

Scenario J: "How is it deployed?"

I run it on a Google Cloud VM - a rented Linux server. Docker Compose starts three containers: PostgreSQL, the Python API, and the Next.js frontend. nginx sits in front as the front door on ports 80/443. A free dpdns domain points to the VM's IP.

To update: pull latest code, run docker compose up -d --build, nginx keeps routing traffic.

Scenario K: "What tech stack did you use?"

- UI: Next.js, React, Tailwind, shadcn/ui
- Charts: Recharts
- API: FastAPI (Python)
- DB: PostgreSQL
- AI: OpenRouter + DeepSeek V4 Flash (open-source)
- Data: ClinicalTrials.gov, openFDA, PubMed
- Infra: Docker, GCP VM, nginx, dpdns

Scenario L: "What are the limitations?"

- Public data only - no proprietary pipelines yet
- Single VM - not auto-scaling or highly available
- First investigation can take 30-60 seconds (live API calls)
- MVP auth - not enterprise SSO or full compliance
- AI outputs still need human verification for high-stakes decisions
- Token costs scale with usage - production needs budgeting

Frame it positively:

Those are intentional MVP boundaries. They keep scope manageable while proving the core workflow.

Part 5: Tough or Surprise Questions

Scenario M: "Did you really build this yourself?"

I built it with AI-assisted development - Cursor/Antigravity helped me write code faster. I designed the architecture, chose the data sources, defined the agent tools and prompts, set up deployment, and tested the flows. Think of it as being the architect and product owner with an AI junior developer, not as copying a tutorial.

Scenario N: "Why pharma / competitive intelligence specifically?"

Because it is a domain where data quality matters more than flashy chat. If the AI invents a trial count, someone could make a bad business decision. That forces good engineering - tool calling, live APIs, server-side metrics. It maps directly to life sciences customers.

Scenario O: "What would you do differently if you started again?"

- Add caching earlier for repeated API calls
- Set up CI/CD from day one instead of manual deploys
- Define the tool schema and dashboard config before building UI
- Write a short architecture doc earlier for onboarding

Scenario P: "Demo walkthrough" (if they ask you to show it live)

Script:

- 1. Open home page - enter a therapy area question
- 2. Point to agent trace: "See it searching ClinicalTrials.gov live"
- 3. Show dashboard: landscape chart, momentum table, white-space section
- 4. Click "Explain these signals" - AI narrates the data
- 5. Ask a follow-up: "Which therapy has the most Phase 3 trials?"
- 6. Generate executive briefing - show structured report
- 7. Optional: run bull/bear debate

If something is slow:

First load hits live APIs - production would cache common queries.

Part 6: Cheat Sheet - Phrases That Sound Informed

- Agentic AI: The LLM picks which tools to call, in a loop, until it has enough data.
- RAG: My MVP uses live API calls instead of a vector DB - RAG on internal docs is a next step.
- Hallucination: Numbers come from code; the LLM only explains them.
- SSE / streaming: Long AI answers stream to the browser so the user sees progress.
- MVP: Minimum viable product - enough to test the workflow, not everything enterprise needs.
- Vibe coding: AI-assisted rapid development with Cursor - fast iteration, human oversight.
- Momentum score: A server-computed ranking from trial activity - not something the model guesses.
- White-space: Areas where few trials exist but differentiation potential is high.

Part 7: Questions YOU Can Ask Them

Shows interest and ties back to your project:

- 1. "Do your customers already use ClinicalTrials.gov manually, or do you integrate trial data?"
- 2. "How do you handle AI governance when outputs could influence business decisions?"
- 3. "Are you investing in agentic workflows or mostly chat/RAG assistants today?"

- 4. "What would make an MVP like this valuable enough to pilot with a real team?"
- 5. "Do you deploy customer workloads on GCP, and would containerized apps fit your architecture?"

Practice out loud once. Pick 3 scenarios most likely for your interview and memorize the opening two sentences of each.